

Oracle AI Database 26ai

Agentic AI Demo

Compute Node Step-by-Step Runbook

SQL + Oracle AI Vector Search + Local Ollama LLM + Tool-Calling Agent

Prepared for the DemoAI Agent environment

Version 1.0

Scope: This runbook covers the agentic demonstration executed from the OCI Compute node. It assumes the native VECTOR datatype and semantic-search database objects are already working in AI_DEMO.

Cost boundary: The runbook uses a local Ollama model on the Compute node and the ONNX embedding model inside Oracle. It does not call a paid external LLM endpoint. The Compute node can still incur charges while running.

1. What this demonstration proves

- A user can ask a business question in ordinary language.
- The agent decides which tool to use instead of following a single hard-coded query.
- The selected tool generates a query embedding and searches Oracle AI Database 26ai.
- Oracle returns the most relevant enterprise data using vector similarity.
- The local Ollama LLM turns the retrieved evidence into a concise grounded answer.
- The response can include the source file, chunk, similarity score, and tool execution details.

Layer	Component	Responsibility
User interface	Terminal or Streamlit	Accepts the user question and displays the response
Agent	basic_agent.py / app agent module	Selects tools, manages the conversation, and builds the final prompt
Local LLM	Ollama with qwen2.5:3b	Performs tool selection and produces the grounded answer
Retrieval tool	search_documents.py / retrieval module	Generates query embeddings and retrieves nearest chunks
Database	Oracle AI Database 26ai	Stores documents, VECTOR embeddings, metadata, and vector indexes
Embedding model	In-database ONNX MiniLM model	Converts text into 384-dimensional vectors

Key distinction: The embedding model performs semantic encoding. The Ollama model performs language reasoning and answer generation. They are different models with different jobs.

2. Environment assumptions

Item	Expected value
Compute host	Oracle Linux Compute instance (for example, toronto or dallas)
Application directory	/opt/oracle-ai-agent
Python environment	/opt/oracle-ai-agent virtual environment or project .venv
Database schema	AI_DEMO
Wallet directory	/opt/oracle-ai-agent/wallet
Database service	Use the exact service from wallet/tnsnames.ora, such as demoai_high or saturn_high
Ollama endpoint	http://127.0.0.1:11434
Ollama model	qwen2.5:3b (or the installed local model configured in .env)
Embedding model	ALL_MINILM_L12_V2 or the exact model visible in ALL_MINING_MODELS
Sample data	Document chunks and embeddings already loaded in AI_DEMO

Security: Do not display or commit database passwords, OCI auth tokens, wallet passwords, API keys, private SSH keys, or the contents of .env.

3. Pre-demo startup and validation

Step 1 - Connect to the Compute node

Purpose: Open an SSH session and confirm you are on the intended demo host.

Run as: Mac or Cloud Shell user, then `opc`

```
ssh -i ~/.ssh/id_rsa opc@<COMPUTE_PUBLIC_IP>
hostname
whoami
date
```

Expected result: The prompt shows the expected host and the operating-system user is `opc`.

Presentation talking point: The agent is running on an OCI Compute node; it is not running inside Database Actions.

Step 2 - Become root only when required

Purpose: Use root for service management and operating-system checks, not for normal application execution.

Run as: `opc`

```
sudo -i
whoami
exit
```

Expected result: `sudo -i` returns a root shell; `exit` returns to `opc`.

Presentation talking point: Separating root administration from application execution reduces accidental permission and ownership problems.

Step 3 - Confirm operating system resources

Purpose: Verify the node has enough memory, disk, and swap for Ollama and the Python agent.

Run as: `opc`

```
cat /etc/os-release
free -h
df -h
swapon --show
uptime
```

Expected result: Disk is not full and available memory plus swap is sufficient. On a small node, some swap is strongly recommended.

Presentation talking point: A local LLM is the largest consumer of memory in this architecture.

Step 4 - Enter the application directory

Purpose: Work from the consistent application root so relative paths, wallet files, and environment variables resolve correctly.

Run as: opc

```
cd /opt/oracle-ai-agent
pwd
ls -la
```

Expected result: The current path is /opt/oracle-ai-agent and the project files are visible.

Presentation talking point: A predictable directory structure makes the demo reproducible on another Compute node.

Step 5 - Activate the Python virtual environment

Purpose: Use the project-specific Python interpreter and installed dependencies.

Run as: opc

```
source /opt/oracle-ai-agent/bin/activate

which python
python --version
python -m pip --version
```

Expected result: which python points under /opt/oracle-ai-agent and the expected Python version is displayed.

Presentation talking point: The virtual environment isolates the demo from operating-system Python packages.

Step 6 - Validate required Python imports

Purpose: Confirm the database driver, HTTP client, environment loader, and data libraries load successfully.

Run as: opc

```
python - <<'PY'
import oracledb
import requests
import dotenv
import pandas

print('python-oracledb:', oracledb.__version__)
print('requests:', requests.__version__)
print('pandas:', pandas.__version__)
print('Imports: PASS')
PY
```

Expected result: The command prints package versions and Imports: PASS.

Presentation talking point: This check catches environment problems before the live demo begins.

Step 7 - Check the private configuration file

Purpose: Confirm .env exists and contains the required variable names without displaying secret values.

Run as: opc

```
test -f .env && echo '.env: FOUND' || echo '.env: MISSING'
```

```
grep -E '^(DB_USER|DB_DSN|DB_WALLET_LOCATION|OLLAMA_URL|OLLAMA_MODEL)= ' .env | sed 's/=.*/=<configured>/'
```

Expected result: All required configuration keys appear as configured.

Presentation talking point: Secrets are loaded at runtime and remain outside source control.

Typical .env keys: DB_USER, DB_PASSWORD, DB_DSN, DB_WALLET_LOCATION, DB_WALLET_PASSWORD, OLLAMA_URL, OLLAMA_MODEL, and optionally TOP_K.

Step 8 - Validate the Autonomous Database wallet

Purpose: Confirm the wallet and network alias files are present.

Run as: opc

```
ls -l wallet
```

```
grep -i '_high|_medium|_low' wallet/tnsnames.ora
```

Expected result: Wallet files such as tnsnames.ora and sqlnet.ora are present, and the configured DB_DSN matches one listed service.

Presentation talking point: The wallet provides secure TLS connectivity from the Compute node to Autonomous Database.

Step 9 - Test database connectivity

Purpose: Open a lightweight connection as AI_DEMO and verify the database version and service.

Run as: opc

```
python scripts/test_database.py
```

Expected result: Database connection: PASS, current user AI_DEMO, and the intended 26ai service are displayed.

Presentation talking point: The agent must connect as the application schema rather than ADMIN.

If the script is unavailable: Use the project's existing database test module or run a one-line python-oracledb connection test. Do not put credentials directly into shell history.

Step 10 - Validate vector and model objects

Purpose: Confirm the document table contains rows, embeddings are populated, and the ONNX model is visible.

Run as: opc

```
python - <<'PY'
from app.database import get_connection

sql = """
SELECT COUNT(*) AS chunk_count,
       COUNT(embedding) AS embedded_count
FROM documents
"""

with get_connection() as conn:
    with conn.cursor() as cur:
        cur.execute(sql)
        print('Documents / embeddings:', cur.fetchone())

        cur.execute("""
            SELECT owner, model_name, mining_function, algorithm
            FROM all_mining_models
            ORDER BY owner, model_name
        """)
        for row in cur:
            print(row)
PY
```

Expected result: The document and embedding counts are nonzero, and the intended embedding model is listed.

Presentation talking point: The database already contains the searchable knowledge base before the agent starts.

Adjust object names: If your repository uses DEMO_DOCUMENTS, DOCUMENT_CHUNKS, or another table name, replace DOCUMENTS in the validation query with the actual object.

4. Start and validate the local LLM

Step 11 - Check the Ollama service

Purpose: Confirm the local model server is running and listening on the loopback interface.

Run as: opc

```
sudo systemctl status ollama --no-pager  
  
ss -lntp | grep 11434 | | true
```

Expected result: The service is active and port 11434 is listening.

Presentation talking point: The LLM stays local to the Compute node; the database is not hosting the generative model.

Step 12 - Start Ollama if necessary

Purpose: Start and enable the service when it is inactive.

Run as: opc

```
sudo systemctl enable --now ollama  
sudo systemctl status ollama --no-pager
```

Expected result: The Ollama service shows active (running).

Presentation talking point: This is a service-management step and therefore uses sudo.

Step 13 - Confirm the required model is installed

Purpose: List local models and verify the configured model name.

Run as: opc

```
ollama list
```

Expected result: qwen2.5:3b, or the model specified by OLLAMA_MODEL, appears in the list.

Presentation talking point: The local model is downloaded once and then reused without an external inference charge.

Step 14 - Perform a direct model smoke test

Purpose: Verify the local LLM can produce a short response before involving the agent.

Run as: opc

```
ollama run qwen2.5:3b 'Explain Oracle Data Guard in two sentences.'
```

Expected result: A concise response is returned and the command completes.

Presentation talking point: This isolates LLM health from database and agent logic.

Step 15 - Test the Ollama REST endpoint

Purpose: Confirm the Python agent can reach the same HTTP API it will call during tool selection and answer generation.

Run as: opc

```
curl -s http://127.0.0.1:11434/api/chat -H 'Content-Type: application/json' -d '{
  "model": "qwen2.5:3b",
  "stream": false,
  "messages": [{"role": "user", "content": "Reply with OLLAMA REST PASS"}]
}'
```

Expected result: JSON is returned containing OLLAMA REST PASS or an equivalent response.

Presentation talking point: The terminal client and the agent use the same local model server through different interfaces.

5. Validate retrieval before running the agent

Step 16 - Run the semantic retrieval tool directly

Purpose: Test database retrieval independently from LLM reasoning.

Run as: opc

```
python search_documents.py 'Which employee works in transportation?'
```

Expected result: The tool returns the relevant document chunk, source filename, and a similarity or distance value. In the sample data, the transportation engineer record should rank highly.

Presentation talking point: This proves Oracle semantic search before the agent is allowed to reason over the result.

Step 17 - Run a second retrieval query

Purpose: Verify the search works for a different concept and does not depend on one memorized phrase.

Run as: opc

```
python search_documents.py 'Who is the civil engineer?'
```

Expected result: The tool returns the chunk identifying the civil engineer, such as John Smith, with source metadata.

Presentation talking point: Semantic retrieval works even when the wording differs from the document text.

Step 18 - Inspect the retrieved evidence

Purpose: Confirm the tool exposes enough information to support a grounded final answer.

Run as: opc

```
# Review the retrieval output for:  
# - source filename  
# - chunk identifier  
# - source text  
# - similarity or distance score  
# - number of returned matches
```

Expected result: The evidence is readable and directly supports the expected answer.

Presentation talking point: The agent should answer from retrieved evidence, not from unsupported model memory.

Distance versus similarity: If the tool reports cosine distance, lower is better. If it reports similarity, higher is better. Label the metric accurately in the output and presentation.

6. Run the agentic workflow

Step 19 - Run the command-line agent

Purpose: Start the full agent, allowing the local LLM to choose and invoke the document-search tool.

Run as: opc

```
python basic_agent.py 'Who is the civil engineer?'
```

Expected result: The console shows that the search_documents tool was selected, displays or logs retrieved evidence, and returns a grounded answer.

Presentation talking point: Unlike a fixed search script, the agent decides whether and how to use a tool.

Step 20 - Run the transportation question through the agent

Purpose: Demonstrate a second tool invocation and grounded answer.

Run as: opc

```
python basic_agent.py 'Which employee works in transportation?'
```

Expected result: The agent identifies the transportation employee and cites the retrieved chunk or source file.

Presentation talking point: The LLM is orchestrating the workflow, but Oracle remains the source of enterprise truth.

Step 21 - Test an out-of-scope question

Purpose: Verify the agent does not invent an answer when the database does not contain supporting evidence.

Run as: opc

```
python basic_agent.py 'Which employee is a marine biologist?'
```

Expected result: The agent states that the supplied documents do not contain enough information, or returns no confident match.

Presentation talking point: A trustworthy enterprise agent should be able to say that the evidence is insufficient.

Step 22 - Run a multi-turn or follow-up question if supported

Purpose: Show whether the agent can use recent context while continuing to ground factual claims.

Run as: opc

```
python basic_agent.py 'Who is the civil engineer, and what is that person responsible for?'
```

Expected result: The response combines the retrieved identity and responsibilities only when both are present in the evidence.

Presentation talking point: Agentic behavior combines planning, retrieval, and response generation rather

than only nearest-neighbor ranking.

Do not overclaim: The agent is agentic because it selects and invokes tools. It is not fully autonomous unless it can plan multiple steps, maintain state, and execute actions without a fixed one-tool path.

7. Run the Streamlit interface

Step 23 - Start the web application

Purpose: Launch the user-facing interface from the activated environment.

Run as: opc

```
streamlit run app/main.py --server.address 0.0.0.0 --server.port 8501
```

Expected result: Streamlit displays a local and network URL and remains running.

Presentation talking point: The UI changes the experience, but the underlying agent, retrieval, database, and model remain the same.

Step 24 - Open the required network path

Purpose: Only when remote browser access is required, confirm OCI and host firewalls allow the selected port from an authorized source.

Run as: opc

```
sudo firewall-cmd --list-ports  
  
# If required and approved:  
sudo firewall-cmd --permanent --add-port=8501/tcp  
sudo firewall-cmd --reload
```

Expected result: Port 8501 is available only to the intended source range or through an SSH tunnel.

Presentation talking point: For a secure demo, prefer an SSH tunnel instead of exposing Streamlit to the entire internet.

Preferred access: From your Mac, use: `ssh -L 8501:127.0.0.1:8501 -i ~/.ssh/id_rsa opc@<PUBLIC_IP>`, then browse to `http://localhost:8501`.

Step 25 - Run the live UI questions

Purpose: Use a short, repeatable sequence that demonstrates tool selection, retrieval, and grounded answers.

Run as: opc / browser

```
Question 1: Who is the civil engineer?  
Question 2: Which employee works in transportation?  
Question 3: Which employee is a marine biologist?
```

Expected result: The first two questions return evidence-based answers. The third demonstrates appropriate uncertainty.

Presentation talking point: The same application handles success cases and missing-evidence cases.

Step 26 - Stop the interactive application

Purpose: Exit cleanly after the demo so the terminal and port are released.

Run as: opc

```
# In the Streamlit terminal:  
Ctrl+C
```

Expected result: Streamlit exits and port 8501 is no longer listening.

Presentation talking point: Clean shutdown is part of a repeatable demonstration.

8. Operational checks and troubleshooting

Symptom	Likely cause	Correction
SSH times out	OCI route, security rule, or VM boot issue	Verify public IP, Internet Gateway route, port 22 rule, then reboot or use serial console
Connection refused after reboot	sshd not yet listening	Wait for boot completion; verify systemctl status sshd through serial console
python command not found	Virtual environment is not active	source /opt/oracle-ai-agent/bin/activate
oracledb import fails	Dependencies installed in another interpreter	Use python -m pip install -r requirements.txt inside the active environment
Database connection fails	Wrong DSN, wallet path, password, or network access	Validate .env, wallet/tnsnames.ora, and scripts/test_database.py
Connected to 19c	Wrong database/service	Open the 26ai Autonomous Database service and verify v\$version
Model not found	Wrong model name or schema visibility	Query ALL_MINING_MODELS and use the exact accessible model name
Ollama API fails	Service stopped or model missing	systemctl start ollama; ollama list; test port 11434
Agent hangs	Small VM memory pressure or model load delay	Check free -h, swap, journalctl, and Ollama logs; allow warm-up time
Poor retrieval result	Wrong embeddings, metric, or insufficient sample data	Validate dimension, model consistency, query text, and stored chunks
Agent hallucinates	Prompt does not require evidence or no-result handling	Require answers only from retrieved context and explicitly allow “not found”
Streamlit unreachable	Port blocked or bound only to localhost	Use SSH tunnel or bind 0.0.0.0 with controlled firewall rules

Useful logs and diagnostics

```
# Ollama service logs
sudo journalctl -u ollama -n 100 --no-pager
sudo journalctl -u ollama -f

# Running processes and ports
ps -ef | grep -E 'ollama|streamlit|python' | grep -v grep
ss -lntp | grep -E '11434|8501'

# Resource use
free -h
top

# Application log, if configured
tail -100 /var/log/oracle-ai-agent.log
```

9. Recommended live-demo script

Order	Action	What to say
1	Architecture in one sentence	The local LLM decides which tool to use; Oracle generates/searches embeddings and returns enterprise evidence.
2	Show services	Confirm Ollama and database connectivity.
3	Direct retrieval	Run search_documents.py to prove semantic search independently.

4	Agent invocation	Run basic_agent.py and show the tool decision.
5	Grounded answer	Point to filename, chunk, and score.
6	Missing evidence	Ask the marine-biologist question and show appropriate uncertainty.
7	Optional UI	Repeat the same questions in Streamlit.
8	Close	The LLM does not know the enterprise data until the agent retrieves it from Oracle.

Core message: The database is doing the heavy lifting. Instead of searching text, we are searching meaning. The agent uses those results as evidence before the local LLM answers.

10. Post-demo shutdown and cost control

Step 27 - Stop interactive processes

Purpose: Ensure Streamlit and manually started scripts are no longer running.

Run as: opc

```
pkill -f 'streamlit run app/main.py' || true
ps -ef | grep -E 'streamlit|basic_agent|search_documents' | grep -v grep
```

Expected result: No transient demo processes remain.

Presentation talking point: Stopping the UI does not stop the Compute charge.

Step 28 - Decide whether to stop Ollama

Purpose: Ollama can remain installed while the service is stopped between sessions.

Run as: opc

```
sudo systemctl stop ollama
sudo systemctl status ollama --no-pager
```

Expected result: The service is inactive. The downloaded model remains on disk.

Presentation talking point: Stopping Ollama frees memory but does not stop Compute billing.

Step 29 - Stop the OCI Compute instance

Purpose: Stop the VM through OCI after you finish so standard compute consumption is paused.

Run as: OCI Console or authorized CLI

```
# From OCI CLI on an authorized machine:
oci compute instance action --instance-id <INSTANCE_OCID> --action SOFTSTOP
```

Expected result: The lifecycle state progresses from STOPPING to STOPPED.

Presentation talking point: A Linux shutdown alone is not the same as stopping the instance through OCI. Boot-volume storage can continue to incur charges.

Appendix A - One-command preflight checklist

```
cd /opt/oracle-ai-agent && source /opt/oracle-ai-agent/bin/activate && echo '=== HOST ===' && hostname && date && echo '===  
RESOURCES ===' && free -h && df -h / && echo '=== PYTHON ===' && python --version && echo '=== OLLAMA ===' && curl -s  
http://127.0.0.1:11434/api/tags && echo && echo '=== DATABASE ===' && python scripts/test_database.py
```

Use before a rehearsal: Run the preflight at least 15 minutes before the session so the local model is warm and any database or network issue can be corrected.

Appendix B - Suggested repository mapping

Path	Purpose
app/main.py	Streamlit user interface
app/agent.py or basic_agent.py	Agent orchestration and tool selection
app/retrieval.py or search_documents.py	Oracle vector retrieval tool
app/database.py	Database connection utility
scripts/test_database.py	Connectivity smoke test
scripts/run_agent.sh	Repeatable command-line launcher
systemd/oracle-ai-agent.service	Optional managed service definition
docs/	Architecture, build guide, and demo script
sql/demo/	Vector datatype and semantic-search SQL demonstrations

Appendix C - Success criteria

- Compute host is reachable and stable.
- Python environment and imports pass.
- AI_DEMO connects to the intended 26ai service.
- Document chunks and embeddings are present.
- Ollama model responds locally.
- Direct semantic retrieval returns the expected source chunk.
- The agent explicitly invokes the retrieval tool.
- The final answer is supported by the retrieved evidence.
- The agent handles missing evidence without fabricating an answer.
- The Compute instance is stopped after the demo when no longer required.